

NOVAVISION PACKAGE DOCUMENTATION

ImageStack

Canlı video akışından son kareleri sınırlı bir tamponda tutan, yeniden boyutlandıran ve görsel zaman çizelgesi üreten NovaVision bileşeni.

Belge	Teknik paket raporu
Paket	ImageStack
Sürüm durumu	Alpha ortamında doğrulandı
Tarih	31 Ağustos 2026

DigiNova / NovaVision

Repository: Abd2023/novavision-image-stack

İçindekiler

1. Genel Bakış	3
Paketin amacı ve ne yaptığı	3
Temel özellikler	3
Desteklenen sınıflar / modeller / tipler	4
2. Mimari ve Teknolojiler	5
Teknoloji Stack'i	5
Her teknolojinin rolü ve kullanımı (kart formatında)	5
Proje yapısı (tree formatında)	6
3. Executor'lar ve Çalışma Modları	7
ImageStack (Tam path: src/executors/ImageStack.py)	7
4. Girdi (Input) Parametreleri	8
4.1 inputImage (Pydantic Model: InputImage)	8
5. Konfigürasyon (Config) Parametreleri	9
5.1 StackSize	9
5.2 ResolutionWidth	9
5.3 ResolutionHeight	9
5.4 ClearBuffer (True / False - Dropdown)	10
6. Çıktı (Output) Parametreleri	11
6.1 outputImages (Pydantic Model: OutputImages)	11
6.2 outputPreview (Pydantic Model: OutputPreview)	11
6.3 outputData (Pydantic Model: OutputData)	12
7. Veri Modelleri	13
PackageModel hiyerarşisi (ASCII tree)	13
Request / Response akışları (her executor için)	13
8. Metodoloji ve Algoritmalar	15
8.1 Görüntü Decode ve Normalizasyonu	15
8.2 En-Boy Oranlı Resize ve JPEG Sıkıştırma	15
8.3 Düşüme Özel Sınırlı FIFO Tamponu	15
8.4 Stack Preview Temas Sayfası	16
8.5 Karmaşıklık ve Bellek Davranışı	16
8.6 Doğrulama ve Kabul Sonuçları	17
8.7 Avantajlar	17

1. Genel Bakış

Paketin amacı ve ne yaptığı

`ImageStack`, canlı video akışından gelen en güncel görüntüleri sınırlı bir bellek kuyruğunda tutan NovaVision bileşenidir. Her çalışma çevriminde gelen görüntünün ilk karesi alınır, gerekirse en-boy oranı korunarak küçültülür, JPEG olarak sıkıştırılır ve en yeni kare listenin başında olacak şekilde saklanır.

Paket bir yapay zekâ modeli çalıştırmaz. Bir video analitiği akışında olay öncesi/sonrası görsel bağlam oluşturmak, son kareleri toplu göstermek ve sonraki bileşenlere yakın zamanlı bir görüntü listesi vermek için kullanılan yardımcı bir bileşendir.

Alpha ortamında doğrulanan örnek kullanım:

```
Video Feed —> Limit Rate —> Image Stack —> Image View
|
|— outputImages: makine tarafından işlenecek kare listesi
|— outputPreview: tek görselde son karelerin zaman çizelgesi
|— outputData: mevcut kare sayısı
```

Önerilen demo ayarları `Limit Rate = Time / 1 second`, `Stack Size = 6`, `Resolution = 640 x 360` ve `Clear Buffer = False` değerleridir. Bu yapı, son altı örneklenmiş kareyi anlaşılır bir zaman çizelgesi olarak `Image View` üzerinde gösterir.

Temel özellikler

- [OK] En yeni kare önce olacak şekilde sınırlı FIFO görüntü geçmişi
- [OK] `StackSize` değiştiğinde mevcut en yeni kareleri koruma
- [OK] En-boy oranını koruyan küçültme ve görüntüyü büyütme
- [OK] JPEG kalite 75 ile bellek kullanımını azaltma
- [OK] NovaVision `Image View` ile uyumlu tek-görsel temas sayfası (`outputPreview`)
- [OK] Liste tüketicileri için orijinal kare koleksiyonu (`outputImages`)
- [OK] Mevcut kare sayısını veren sayısal çıkış (`outputData`)
- [OK] `ClearBuffer` için `False` → `True` yükselen-kenar temizleme davranışı
- [OK] Aynı düğümde farklı `flowUID` değerleri arasında devam eden tampon
- [OK] Düğüm bazlı bağımsız durum: `matchedID`, yoksa kararlı `uID`
- [OK] Yerel test ve NovaVision Suite doğrudan-script çalışma yolları
- [OK] Yeniden boyutlandırılmış görüntü boyutlarını doğru raporlayan metadata

Desteklenen sınıflar / modeller / tipler

Model / Tip	Amaç	Temel alanlar	Not
<code>InputImage</code>	Akıştan görüntü alma	<code>value, type</code>	Tek <code>Image</code> veya <code>Image</code> listesi; listede ilk eleman kullanılır
<code>StackSize</code>	Tampon kapasitesi	<code>value: int</code>	1-64, varsayılan 10
<code>ResolutionWidth</code>	Maksimum saklama genişliği	<code>value: int</code>	64-1920, varsayılan 1920
<code>ResolutionHeight</code>	Maksimum saklama yüksekliği	<code>value: int</code>	64-1080, varsayılan 1080
<code>ClearBuffer</code>	Tampon temizleme seçeneği	<code>False / True</code>	Yalnızca <code>False</code> → <code>True</code> geçişinde bir kez temizler
<code>OutputImages</code>	Son kareler	<code>value: List[Image]</code>	En yeni kare önce, dış socket tipi <code>list</code>
<code>OutputPreview</code>	Görsel zaman çizelgesi	<code>value: Image</code>	Dış socket tipi <code>object</code> ; <code>Image View</code> ile uyumlu
<code>OutputData</code>	Kare sayısı	<code>value: int</code>	Dış socket tipi <code>number</code>

Desteklenen görüntü veri biçimleri:

- Base64 kodlanmış JPEG/PNG benzeri görüntü verisi
- Ham `bytes` ve geçerli `shape_key`
- Yerel test yolunda NumPy `ndarray`
- Suite çalışma yolunda Redis anahtarlı NovaVision görüntüsü (`r_key`)

2. Mimari ve Teknolojiler

Teknoloji Stack'i

Teknoloji	Sürüm / Aralık	Kullanım
Python	3.12+	Executor, servis ve test kodu
OpenCV	>=4.10, <5	Görüntü decode, resize, JPEG encode ve preview üretimi
NumPy	>=1.26, <3	Kare ve <code>shape_key</code> dizileri
Pydantic	>=1, <3	NovaVision PackageModel, Request/Response ve parametre doğrulaması
NovaVision SDK	Kurulu Suite sürümü	<code>Component</code> , <code>Image</code> , <code>Package</code> , <code>helper</code> ve <code>executor</code> çalışma zamanı
Pytest	>=8, <9	Model ve executor regresyon testleri
Docker	Python 3.12 slim	Geliştirme ve üretim image tanımları

Her teknolojinin rolü ve kullanımı (kart formatında)

Python - Paket kontrol akışını, durum yönetimini, NovaVision çalışma zamanı uyumluluğunu ve hata mesajlarını sağlar.

OpenCV - `cv2.imdecode` ile görüntüyü açar, `cv2.resize(..., INTER_AREA)` ile küçültür, `cv2.imencode` ile JPEG kalite 75 çıktısı üretir ve temas sayfası üzerindeki çerçeve/etiketleri çizer.

NumPy - Görüntü matrisini taşır. Ayrıca `(height, width, channels)` şekli `int64` byte dizisine dönüştürülerek `shape_key` üretilir.

Pydantic ve NovaVision SDK modelleri - Suite formunun, socket'lerin ve çalışma zamanı paketinin aynı `PackageModel.configs.executor.value.value` yolu üzerinden doğrulanmasını sağlar. SDK bulunmadığında yerel fallback modelleri kullanılır.

Deque tabanlı durum yönetimi - Python `collections.deque(maxlen=StackSize)` eski kareleri otomatik düşürür. `appendleft` sayesinde çıktı her zaman en yeni kareden en eski kareye sıralanır.

Pytest - FIFO sırası, kapasite değişimi, resize, metadata, preview, temizleme, düğüm izolasyonu ve değişken `flowUID` davranışlarını doğrular.

Docker - `Dockerfile.prod` yalnızca üretim bağımlılıklarını, `Dockerfile.dev` ise test bağımlılıklarını da yükler. Her iki image'ın varsayılan komutu `python service.py` şeklindedir.

Proje yapısı (tree formatında)

```
novavision-image-stack/
├── apps/
│   ├── run_sample_client.py      # Dört karelik yerel FIFO örneği
│   └── sample_request.json      # Örnek NovaVision istek gövdesi
├── notebooks/
│   └── README.md                # Notebook kullanım notu
├── src/
│   ├── executors/
│   │   └── ImageStack.py        # Ana executor ve Suite script entrypoint
│   └── models/
│       └── PackageModel.py      # Tek kanonik PackageModel ve socket modelleri
├── tests/
│   ├── test_executor.py         # Davranış/regresyon testleri
│   └── test_package_model.py    # Şema ve model testleri
├── Dockerfile.dev
├── Dockerfile.prod
├── DOCUMENTATION.md            # Bu teknik rapor
├── LICENSE                     # Apache-2.0
├── NOTICE                     # Roboflow davranış referansı bildirimi
├── README.md
├── pyproject.toml
├── requirements.dev.txt
├── requirements.prod.txt
└── service.py                  # Executor kayıt ve bootstrap kontrolü
```

3. Executor'lar ve Çalışma Modları

`ImageStack` (Tam path: `src/executors/ImageStack.py`)

Ana sınıf `ImageStack`, NovaVision SDK mevcutsa SDK `Component` sınıfından, yerel çalışmada ise uyumluluk fallback sınıfından türetilir.

Temel yaşam döngüsü:

1. `bootstrap()` ortak tamponları, clear durumunu ve kilidi oluşturur.
2. `__init__(request, bootstrap)` isteği ve aynı process içindeki ortak durumu bağlar.
3. `run()` input/config değerlerini okur ve sınırları doğrular.
4. Görüntü decode edilir, gerekiyorsa küçültülür ve JPEG'e çevrilir.
5. Kare kararlı düğüm anahtarına ait deque'in başına eklenir.
6. `outputImages`, `outputPreview` ve `outputData` oluşturulur.
7. Yerel çağrıda `ImageStackResponse`, Suite çağrısında tam `PackageModel` döndürülür.

Bootstrap yapısı:

```
@staticmethod
def bootstrap(config=None):
    return {
        "status": "ready",
        "image_stack_buffers": {},
        "image_stack_clear_seen": {},
        "image_stack_lock": threading.RLock(),
    }
```

Çalışma modları:

Mod	İstek tipi	Dönüş tipi	Kullanım
Yerel model modu	<code>ImageStackRequest</code>	<code>ImageStackResponse</code>	Unit test ve örnek istemci
Suite runtime modu	SDK request / sözlük payload	<code>PackageModel</code>	NovaVision akışı
Doğrudan script modu	<code>ImageStack.py <nodeUID></code>	SDK MQTT executor döngüsü	Suite container çalışma şekli

Suite script entripoint:

```
if __name__ == "__main__":
    from sdks.novavision.src.helper.executor import Executor
    Executor(sys.argv[1]).run()
```

Durum process belleğindedir. Servis yeniden başlatılır veya paket yeniden deploy edilirse tampon sıfırlanır. Aynı process ve aynı düğüm kimliği içinde yeni executor instance'ları oluşturulsa da tampon devam eder.

4. Girdi (Input) Parametreleri

4.1 `inputImage` (Pydantic Model: `InputImage`)

Özellik	Değer
NovaVision adı	<code>inputImage</code>
Suite başlığı	<code>Image</code>
Dış socket tipi	<code>list</code> veya runtime değerine göre <code>object</code>
Zorunluluk	Evet; boş değer hata üretir
Kabul edilen değer	Bir <code>Image</code> veya <code>List[Image]</code>
Liste davranışı	Yalnızca ilk görüntü işlenir

Temel model:

```
class InputImage(NovaVisionInput):  
    name = "inputImage"  
    value: Union[List[NovaVisionImage], NovaVisionImage]  
    type: str = "list"
```

Örnek istek görüntüsü:

```
{  
  "name": "inputImage",  
  "value": [  
    {  
      "name": "Image_123",  
      "value": "<base64>",  
      "type": "Image",  
      "uID": "frame-123",  
      "mimeType": "image/jpg",  
      "encoding": "base64",  
      "r_key": "",  
      "shape_key": "<base64 int64 shape>",  
      "timestamp": 1788190210.958013,  
      "metadata": {  
        "frame_index": 1,  
        "video_fps": 24,  
        "width": 1280,  
        "height": 720  
      }  
    }  
  ],  
  "type": "list"  
}
```

Hata durumları:

- `inputImage` yok veya liste boşsa: `inputImage is required.`
- Base64 geçersizse: `The input image value is not valid base64 data.`
- Ham byte uzunluğu `shape_key` ile uyuşmuyorsa açıklayıcı decode hatası
- Boş ya da 2D/3D olmayan NumPy karelerinde doğrulama hatası
- Redis anahtarlı görüntü Suite helper ile alınamazsa Redis yükleme hatası

5. Konfigürasyon (Config) Parametreleri

5.1 `StackSize`

Alan	Değer
Tip / UI alanı	<code>number / textInput</code>
Varsayılan	10
Minimum	1
Maksimum	64
Amaç	Bellekte tutulacak maksimum kare sayısı

Kapasite dolunca deque en eski kareyi otomatik olarak çıkarır. Değer çalışma sırasında küçültülürse mevcut listenin en yeni `StackSize` kadar karesi korunur; büyütülürse mevcut kareler korunur ve yeni kapasiteye kadar büyümeye devam eder.

5.2 `ResolutionWidth`

Alan	Değer
Tip / UI alanı	<code>number / textInput</code>
Varsayılan	1920
Minimum	64
Maksimum	1920
Amaç	Saklanan tek karenin maksimum genişliği

Bu değer zorunlu çıkış genişliği değil, üst sınırdır. Görüntü daha küçükse büyütülmez. En-boy oranı nedeniyle gerçek genişlik bu değerden küçük olabilir.

5.3 `ResolutionHeight`

Alan	Değer
Tip / UI alanı	<code>number / textInput</code>
Varsayılan	1080
Minimum	64
Maksimum	1080
Amaç	Saklanan tek karenin maksimum yüksekliği

Örnekler:

Kaynak	Ayar	Gerçek çıkış
1280×720 (16:9)	640×360	640×360

Kaynak	Ayar	Gerçek çıkış
1280×720 (16:9)	640×480	640×360
640×480 (4:3)	640×360	480×360
320×180	640×360	320×180 (upscale yapılmaz)

5.4 `ClearBuffer` (True / False - Dropdown)

Alan	Değer
Tip / UI alanı	<code>object / dropdownlist</code>
Varsayılan	<code>False</code>
Seçenekler	<code>False, True</code>
Amaç	Mevcut düğüm tamponunu bir kez temizlemek

Temizleme yükselen kenar ile çalışır:

```
False -> True : tampon temizlenir, mevcut kare ilk kare olarak eklenir
True  -> True : tekrar temizlenmez, kare birikmeye devam eder
True  -> False : temizleme kolu yeniden hazırlanır
False -> True  : ikinci kez temizlenir
```

Suite üzerinde temizledikten sonra değeri tekrar `False` yapıp kaydetmek gerekir. Aksi halde sonraki `True` seçimi yeni bir geçiş oluşturmaz.

6. Çıktı (Output) Parametreleri

6.1 `outputImages` (Pydantic Model: `OutputImages`)

Alan	Değer
Suite başlığı	Images
Dış socket tipi	list
İç değer	List[NovaVisionImage]
Dinleme	continuous
Dal	forward
Sıra	En yeni → en eski

Her kare JPEG kalite 75 ve Base64 formatında döner. `shape_key`, gerçek JPEG matris şeklini (`height`, `width`, `channels`) olarak Base64 kodlanmış `int64` dizisi biçiminde taşır.

640×360 ayarında 1280×720 kaynaktan beklenen metadata:

```
{
  "name": "outputImages",
  "type": "Image",
  "mimeType": "image/jpg",
  "encoding": "base64",
  "shape_key": "<decodes to [360, 640, 3]>",
  "metadata": {
    "width": 640,
    "height": 360,
    "source_width": 1280,
    "source_height": 720,
    "frame_index": 1,
    "video_fps": 24
  }
}
```

Önemli doğrulama notu: İlk Alpha çıktısında `shape_key` zaten (360, 640, 3) değerini veriyordu; yani JPEG gerçekten küçültülüyordu. Sorun, kaynak metadata içindeki `width=1280` ve `height=720` alanlarının değiştirilmeden kopyalanmasıydı. 73cc61b düzeltmesinden sonra `width/height` gerçek çıkışı, `source_width/source_height` ise orijinal kaynağı gösterir.

6.2 `outputPreview` (Pydantic Model: `OutputPreview`)

Alan	Değer
Suite başlığı	Stack Preview
Dış socket tipi	object
İç değer	Tek NovaVisionImage
Kullanım	Image View veya tek-görsel File Save
Sıra	Temas sayfasında en yeni kare önce

Preview, en fazla dört sütunlu bir temas sayfasıdır. Her hücre sıra numarası ve mevcutsa `frame_index` ile etiketlenir. Arka plan koyu gri, hücre sınırları açık gri olarak çizilir.

Örnek çıktı alanları:

```
{
  "name": "outputPreview",
  "type": "Image",
  "uID": "stack-preview-<newest-frame-uid>",
  "mimeType": "image/jpg",
  "encoding": "base64",
  "stack_count": 6,
  "preview_order": "newest-first"
}
```

Preview'in boyutu tek kare boyutuyla aynı olmak zorunda değildir. Örneğin altı adet 640×360 kare için oluşturulan temas sayfası `shape_key` üzerinden (236, 640, 3) olabilir. Bu değer tüm zaman çizelgesi tuvalinin gerçek boyutudur.

6.3 `outputData` (Pydantic Model: `OutputData`)

Alan	Değer
Suite başlığı	Frame Count
Dış socket tipi	number
İç değer	int
Aralık	1..StackSize (başarılı kare işlemesinden sonra)

Örnek:

```
{
  "name": "outputData",
  "value": 6,
  "type": "number",
  "listen": "continuous",
  "branch": "forward"
}
```

Socket kullanım özeti:

Kaynak çıkışı	Uygun tüketici	Uygun olmayan örnek
<code>outputImages</code>	Görüntü listesi kabul eden bileşen	Tek-görsel Image View
<code>outputPreview</code>	Image View, tek-görsel File Save	Sayısal karşılaştırma
<code>outputData</code>	Sayısal mantık/izleme bileşeni	Görüntü gösterici

7. Veri Modelleri

PackageModel hiyerarşisi (ASCII tree)

```

PackageModel (NovaVisionPackage)
├── type = "component"
├── name = "ImageStack"
├── runtime metadata: uID, flowUID, matchedID, debug, api
├── configs (PackageConfigs)
│   └── executor (ConfigExecutor)
│       ├── name = "ConfigExecutor"
│       ├── type = "executor"
│       ├── field = "dependentDropdownlist"
│       └── value (ImageStackExecutor)
│           ├── name = "ImageStack"
│           ├── type = "object"
│           ├── field = "option"
│           └── value
│               ├── ImageStackRequest
│               │   ├── inputs (ImageStackInputs)
│               │   │   └── inputImage (InputImage)
│               │   └── configs (ImageStackConfigs)
│               │       ├── StackSize
│               │       ├── ResolutionWidth
│               │       ├── ResolutionHeight
│               │       └── ClearBuffer
│               └── ImageStackResponse
│                   └── outputs (ImageStackOutputs)
│                       ├── outputImages (OutputImages)
│                       ├── outputPreview (OutputPreview)
│                       └── outputData (OutputData)

```

Modelin Suite açısından kritik traversals yolu:

```
PackageModel.configs.executor.value.value
```

Form alanları, giriş socket'i ve çıkış socket'leri bu yol üzerinden keşfedilir. `OutputPreview` dış tipinin `object` olması, Suite'teki tek-görsel tüketicilerle bağlantı kurulabilmesi için gereklidir; iç runtime değeri yine standart `Image` nesnesidir.

Request / Response akışları (her executor için)

`ImageStack` akışı:

```

[Video Feed / Client]
|
| ImageStackRequest
| inputImage + StackSize + ResolutionWidth/Height + ClearBuffer
v
[ImageStack.run]
|
+--> parametreleri aç ve doğrula
+--> görüntüyü SDK helper veya yerel decoder ile al
+--> en-boy oranlı resize + JPEG kalite 75
+--> metadata boyutlarını gerçek çıkışa göre düzelt
+--> stable node key ile deque'e appendleft
+--> outputImages listesini oluştur
+--> outputPreview temas sayfasını oluştur
+--> outputData sayısını oluştur
v
[ImageStackResponse veya PackageModel Response]
|
+--> outputImages -> liste tüketicileri
+--> outputPreview -> Image View / File Save
└--> outputData -> sayı tüketicileri

```

Durum anahtarı seçimi:

```
node_uid = matched_id or node_uid or self.uID or "local-node"
```

`flowUID` bir akış çalıştırma/korelasyon kimliğidir ve her karede değişebilir. Bu nedenle tampon anahtarında kullanılmaz. Alpha üzerinde kare sayısının sürekli 1 kalmasına neden olan önceki davranış bu ayırım yapılarak düzeltilmiştir.

8. Metodoloji ve Algoritmalar

Aşağıda görüntü toplama algoritmasının amacı, adımları, pseudo-code'u, karmaşıklığı ve doğrulama yöntemi verilmiştir.

8.1 Görüntü Decode ve Normalizasyonu

- Amaç: NovaVision kaynağından gelen farklı görüntü taşıma biçimlerini tek NumPy kareye dönüştürmek.

- Adımlar:

1. `inputImage` listeyse ilk elemanı seç.
2. Suite SDK `Image.get_frame` helper'ı mevcutsa onu dene.
3. Redis anahtarı varsa görüntüyü bağlı Redis veritabanından al.
4. Base64 ise byte dizisine decode et.
5. Sıkıştırılmış görüntüyse `cv2.imdecode` kullan.
6. Ham bytes ise `shape_key` ile NumPy dizisini yeniden şekillendir.
7. Veri tipini gerekirse `uint8` aralığına kırp.

8.2 En-Boy Oranlı Resize ve JPEG Sıkıştırma

- Amaç: Bellek kullanımını sınırlarken görüntüyü bozmadan hedef maksimum boyutlara sığdırmak.

- Ölçek formülü:

```
scale = min(
    1.0,
    ResolutionWidth / source_width,
    ResolutionHeight / source_height
)
```

- 1.0 üst sınırı nedeniyle küçük görüntüler büyütülmez.

- Pseudo-code:

```
def compress_frame(frame, max_width, max_height):
    source_height, source_width = frame.shape[:2]
    scale = min(1.0, max_width / source_width, max_height / source_height)

    if scale < 1.0:
        width = round(source_width * scale)
        height = round(source_height * scale)
        frame = cv2.resize(frame, (width, height), interpolation=cv2.INTER_AREA)

    jpeg = cv2.imencode(".jpg", frame, [cv2.IMWRITE_JPEG_QUALITY, 75])
    return jpeg, frame.shape
```

- Metadata yöntemi:

- `shape_key` her zaman gerçek sıkıştırılmış kare şeklinden üretilir.
- Kaynak metadata `width/height` içeriyorsa çıkış değerleri gerçek boyutla değiştirilir.
- Resize gerçekleşmişse eski değerler `source_width/source_height` alanlarında korunur.
- `frame_index`, `video_fps`, `timestamp`, `video_path` gibi diğer bilgiler değiştirilmez.

8.3 Düşüme Özel Sınırlı FIFO Tamponu

- Amaç: Son N kareyi düşük maliyetle saklamak ve en yeni kareyi ilk sırada sunmak.

- Adımlar:

- Kararlı anahtar için `matchedID`, ardından `uID` kullan.
- Anahtara ait deque yoksa `deque(maxlen=StackSize)` oluştur.
- Kapasite değiştiyse yeni deque oluştur ve en yeni mevcut kareleri koru.
- Clear yükselen kenarı varsa tamponu temizle.
- Yeni kareyi `appendleft` ile başa ekle.
- `maxlen` aşılsa deque en eski kareyi otomatik düşürür.

- Pseudo-code:

```
with lock:
    buffer = buffers.get(node_key)
    if buffer is None:
        buffer = deque(maxlen=stack_size)
    elif buffer.maxlen != stack_size:
        buffer = deque(list(buffer)[:stack_size], maxlen=stack_size)

    if clear and not previous_clear:
        buffer.clear()

    buffer.appendleft(stored_frame)
    frames = list(buffer)
```

8.4 Stack Preview Temas Sayfası

- Amaç: Suite ortamında görüntü-listesi viewer bulunmasa bile tamponun gerçekten çalıştığını tek `Image View` üzerinde göstermek.

- Adımlar:

- Saklanan JPEG karelerini decode et.
- Sütun sayısını `min(4, frame_count)` olarak belirle.
- Satır sayısını tavan bölme ile hesapla.
- Her kareyi hücreye sığdırırken en-boy oranını koru.
- Koyu arka planlı tuvale thumbnail'i ortalay.
- Hücre sınırını çiz.
- Sıra numarası ve varsa `frame_index` etiketini ekle.
- Tüm tuvali JPEG kalite 75 ile encode et.

8.5 Karmaşıklık ve Bellek Davranışı

$N = StackSize$, tek sıkıştırılmış kare boyutu J , preview piksel sayısı P olmak üzere:

İşlem	Zaman	Bellek
Deque'e kare ekleme	$O(1)$	$O(N \times J)$ toplam tampon
<code>outputImages</code> üretimi	$O(N)$	Base64 serialization maliyeti
Preview üretimi	$O(N + P)$	$O(P)$ tuval ve geçici decode kareleri
Clear	$O(N)$ referans bırakma	Tampon mevcut kareleri serbest bırakır

Maksimum `StackSize=64`, maksimum tek-kare sınırı 1920×1080 ve JPEG kalite 75 değerleri kontrolsüz bellek büyümesini engeller. Gerçek bellek tüketimi sahne karmaşıklığına ve JPEG sıkıştırma oranına bağlıdır.

8.6 Doğrulama ve Kabul Sonuçları

Yerel doğrulama komutları:

```
python -m pytest -q
python service.py
python apps\run_sample_client.py
python -m compileall -q src apps service.py
```

Son doğrulama sonucu:

- [OK] Standalone test ortamı: 16 test geçti
- [OK] Alpha'nın kurulu NovaVision SDK yolu ile: 16 test geçti
- [OK] `service.py: ImageStack: ready`
- [OK] Örnek istemci: 1 -> 2 -> 3, ardından FIFO kapasitesinde en eski karenin düşmesi
- [OK] `shape_key`: 1280×720 giriş ve 640×360 ayarında (360, 640, 3)
- [OK] Metadata: çıkış 640×360, kaynak 1280×720 olarak ayrı alanlarda
- [OK] Alpha canlı akış: altı karelik newest-first temas sayfası görüldü
- [OK] Akış durdurulup tek seferlik `Run Flow` çalıştırıldığında aynı process içindeki önceki tampon korundu ve yeni kare listenin başına eklendi

Önerilen son kullanıcı kabul senaryosu:

```
Video Feed
├─> Fall Detection -> If (olay analizi kolu)
├─> Limit Rate -> Image Stack -> Image View
└─> File Save (isteğe bağlı preview kaydı)
```

Bu senaryoda Image Stack'in kanıtlanan görevi, her saniyeden örneklenen son altı kareyi bir zaman çizelgesinde tutmak ve operatöre olay çevresindeki yakın geçmişi göstermektir. Fall Detection/Notification kolu ayrı bileşenlerin runtime uyumluluğuna bağlıdır; Image Stack kabulü için bu kolun başarılı olması gerekmez.

8.7 Avantajlar

- [OK] Yapay zekâ modelinden bağımsız ve farklı video analitiği senaryolarında tekrar kullanılabilir
- [OK] Tek kare yerine kısa zaman bağlamı sağlar
- [OK] Sınırlı FIFO ve JPEG sıkıştırması ile öngörülebilir bellek kullanımı
- [OK] Suite'in tek-görsel `Image View` bileşeniyle doğrudan görsel doğrulama
- [OK] Kaynak ve işlenmiş çözünürlüğü metadata içinde açıkça ayırır
- [OK] Değişken runtime `flowUID` değerlerinden etkilenmeden düğüm bazında birikir
- [OK] Pydantic şeması ile form, socket ve runtime response sözleşmesi doğrulanır
- [OK] Yerel ve kurulu NovaVision SDK ortamlarında aynı regresyon paketiyle test edilir